
MedRoutes

Relatório Técnico — Tech Challenge Fase 2

Projeto 2 — Otimização de Rotas para Distribuição de Medicamentos e Insumos

FIAP PÓS-TECH — IA para Devs Disciplina integradora da Fase 2

Equipe

Nome	RM
Dhiego Pimenta	RMxxxxxx
Rafael Cardoso de Oliveira	RM373129
<i>Integrante 3</i>	<i>RMxxxxxx</i>
<i>Integrante 4</i>	<i>RMxxxxxx</i>

 Demo em produção: <https://ca-medroutes.ambitiousstone-87ab2c0d.eastus2.azurecontainerapps.io>  Vídeo de demonstração: *(inserir link do YouTube/Vimeo)*  Repositório: <https://github.com/DhiegoPimenta/medroutes-fiap>

1. Contexto e objetivo

Um sistema hospitalar precisa distribuir medicamentos críticos e insumos regulares para múltiplos endereços, utilizando uma frota com capacidade de carga e autonomia limitadas. Trata-se de uma variante do clássico **Travelling Salesman Problem (TSP)** estendida para múltiplos veículos: o **Vehicle Routing Problem (VRP)**.

O **MedRoutes** resolve esse problema com um **Algoritmo Genético (AG) implementado do zero** — sem bibliotecas prontas de computação evolutiva (DEAP, PyGAD) —, visualiza as rotas em um mapa interativo e usa um **LLM (Claude, da Anthropic)** para transformar a saída numérica do otimizador em instruções operacionais, relatórios executivos e respostas em linguagem natural.

Objetivo de otimização: minimizar a distância total percorrida pela frota, respeitando capacidade e autonomia, e **priorizando a entrega de medicamentos críticos**.

Decisões de modelagem realistas:

- Medicamentos críticos devem ser entregues **antes** de insumos regulares na mesma rota.
- Veículos têm **capacidade de carga máxima (kg)** e **autonomia máxima (km)**.
- A frota pode ter **múltiplos veículos** atuando simultaneamente.
- Todas as rotas **partem e retornam ao depósito central**.

2. Modelagem do problema

2.1 Representação cromossômica — *giant tour*

O cromossomo é uma **permutação dos índices de todas as entregas**. Exemplo com 5 entregas e 2 veículos:

```
Cromossomo: [2, 0, 4, 1, 3]
              índices das entregas, na ordem de visita

Decodificação (greedy por capacidade):
  Veículo 0: entrega[2] → entrega[0] → entrega[4]    (até encher a capacidade)
  Veículo 1: entrega[1] → entrega[3]                  (restante)
```

A escolha do *giant tour* (uma única sequência decodificada em múltiplas rotas) é deliberada: permite reutilizar **operadores clássicos e bem estudados de permutação** (Order Crossover, mutação por inversão), que garantem cromossomos sempre válidos — sem genes duplicados ou

ausentes. A alternativa (codificar explicitamente a divisão entre veículos) exigiria operadores sob medida, mais propensos a gerar soluções inválidas e mais difíceis de testar.

2.2 Decodificação *greedy*

A função `decode_chromosome` percorre a permutação e aloca cada entrega ao veículo atual enquanto houver capacidade; quando não cabe, avança para o próximo veículo. Se todos se esgotam, as entregas restantes são forçadas no último veículo, **violando o limite de forma proposital** — essa violação é fortemente penalizada na função fitness, criando pressão seletiva contra soluções inviáveis sem precisar descartá-las (o que reduziria a diversidade genética).

3. Algoritmo Genético

3.1 Operadores

Operador	Implementação	Papel
Seleção por torneio	<code>tournament_selection</code>	Sorteia k indivíduos e escolhe o de menor fitness. Quanto maior k , maior a pressão seletiva.
Crossover OX	<code>order_crossover</code>	Copia um segmento contíguo de um pai e completa com os genes do outro na ordem em que aparecem. Preserva validade da permutação.
Mutação por inversão	<code>inversion_mutation</code>	Inverte um subtrecho aleatório com probabilidade <code>mutation_rate</code> . Introduce diversidade sem quebrar a permutação.
Elitismo	no laço principal	Os melhores indivíduos passam intactos à próxima geração, garantindo que o melhor fitness nunca piore entre gerações (monotonicidade).

3.2 Função fitness multiobjetivo

O problema é de **minimização**: quanto menor o fitness, melhor a solução. Ele combina quatro termos ponderados:

```
fitness = w_dist · distância_total
         + w_prior · penalidade_prioridade
         + w_cap   · kg_excedentes
         + w_auto  · km_excedentes
```

Componente	Peso	Justificativa do peso
Distância total	1,0	Objetivo-base; é a unidade de referência (1 unidade de fitness $\approx$ 1 km).
Penalidade de prioridade	5,0	Restrição leve (soft) : incentiva entregar críticos cedo, mas sem inviabilizar a rota. O valor 5,0 equilibra — pesos muito baixos ignoram a prioridade; muito altos distorcem a distância.
Excesso de capacidade (kg)	1000,0	Restrição forte (hard) : viola um limite físico do veículo. O peso alto (três ordens de grandeza acima da distância) torna qualquer violação dominante no fitness, empurrando a busca para o espaço viável.
Excesso de autonomia (km)	1000,0	Idem — limite físico de alcance.

A modelagem de capacidade e autonomia como **penalização** (e não como rejeição rígida) é uma técnica consagrada em AGs para problemas com restrições: mantém o espaço de busca conexo e permite que soluções temporariamente inviáveis sirvam de "ponte" para regiões viáveis melhores.

3.3 Cálculo de distância — Haversine

As distâncias usam a **fórmula de Haversine** (considera a curvatura da Terra). É uma aproximação em linha reta, adequada à escala urbana do problema; não modela vias reais nem trânsito — ver Limitações (§8).

4. Restrições implementadas

O enunciado exige no mínimo duas restrições adicionais além da ordem de prioridade. O MedRoutes implementa **três**, combinando-as na função fitness:

1. **Prioridade de entrega** — medicamentos críticos (peso 3,0) vs. insumos regulares (peso 1,0); a penalidade cresce conforme um item crítico aparece tarde na rota.
2. **Capacidade de carga (kg)** — soma do peso das entregas de um veículo não pode exceder sua capacidade.
3. **Autonomia (km)** — distância total de uma rota não pode exceder o alcance do veículo.

Além disso, o sistema opera nativamente com **frota de múltiplos veículos**, distribuindo as entregas entre eles na decodificação.

5. Experimentos e resultados

5.1 Metodologia

Todos os experimentos rodam sobre um **cenário sintético fixo e reproduzível** (sem depender de geocodificação real):

- **25 entregas** (1/3 críticas), **4 veículos** (40 kg / 80 km cada), **120 gerações**.
- A semente do **cenário** é fixa (`seed = 7`), separada da semente do **AG** (`random_seed`).
- **Rigor estatístico:** cada configuração é executada com **10 sementes distintas** do AG; reportamos **média ± desvio-padrão**. Isso evita concluir a partir de uma única execução, que seria sensível à sorte de uma semente específica.

Variamos três hiperparâmetros entre as configurações: tamanho da população, taxa de mutação e tamanho do torneio (pressão seletiva).

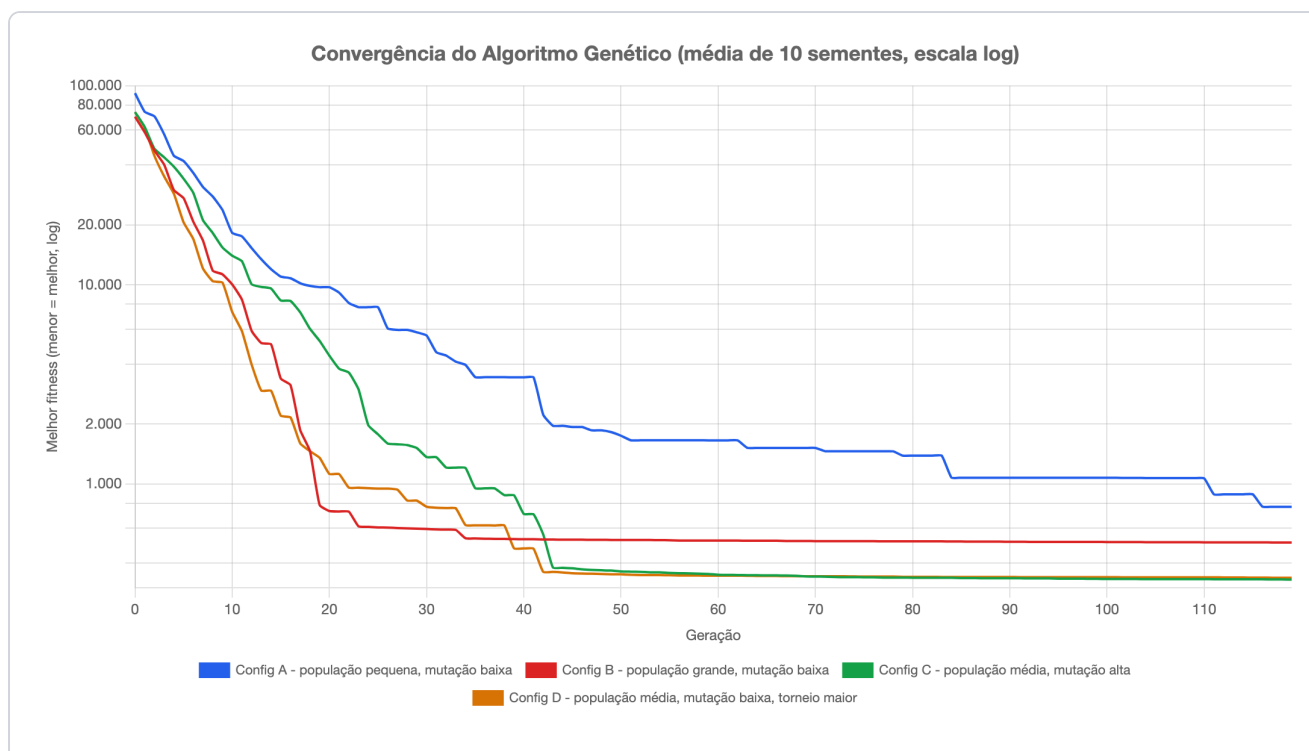
5.2 Baselines de referência

Para situar a qualidade do AG, comparamos com duas abordagens não evolutivas sobre o mesmo cenário:

Abordagem	Distância total
Rota aleatória (média de 10 sementes)	443,79 km
Heurística do vizinho-mais-próximo (<i>nearest neighbor</i>)	236,48 km

5.3 Resultados das configurações do AG

Config	Pop.	Mutação	Torneio	Fitness (média ± dp)	Distância km (média ± dp)	Tempo (s)
A — pop. pequena, mut. baixa	30	0,05	3	767,3 ± 920,6	281,1 ± 12,2	0,133
B — pop. grande, mut. baixa	150	0,05	3	507,4 ± 575,7	238,5 ± 13,7	0,691
C 🌟 — pop. média, mut. alta	80	0,40	3	330,5 ± 10,8	240,9 ± 12,9	0,390
D — pop. média, torneio maior	80	0,10	6	337,6 ± 21,5	246,1 ± 20,3	0,379



5.4 Análise

Config C (população 80, mutação 0,40) é a melhor — e a mais confiável. Não apenas tem o menor fitness médio (330,5), como o **menor desvio-padrão ($\pm 10,8$)**. Esse segundo ponto só é visível porque rodamos múltiplas sementes: a configuração não é boa por sorte, é **consistentemente** boa.

A mutação alta vence a população grande. A Config C (mutação 0,40) supera a Config B (população 150, mutação 0,05) com metade do custo computacional (0,39 s vs. 0,69 s). Diversidade introduzida por mutação foi mais eficaz para escapar de ótimos locais do que diversidade obtida por população maior com pouca mutação.

A Config A é uma armadilha estatística. Seu fitness médio (767) já é ruim, mas o desvio-padrão de **920** é o achado mais importante: a configuração é **instável**. População pequena com mutação baixa fica presa em ótimos locais de qualidade imprevisível — em algumas sementes converge razoavelmente, em outras estaciona muito cedo. Reportar apenas uma execução esconderia esse risco. Na curva de convergência (escala logarítmica), a Config A é a linha que estabiliza num patamar alto e nunca desce.

O AG não é o melhor em distância pura — e isso é esperado. O vizinho-mais-próximo (236,48 km) percorre menos quilômetros que a melhor configuração do AG (Config C: 240,9 km). Isso **não** é um defeito: o vizinho-mais-próximo minimiza *apenas* distância, enquanto o AG otimiza um objetivo **multicritério** que também penaliza entregar medicamentos críticos tardiamente. O AG aceita percorrer ~4 km a mais para atender melhor à prioridade clínica — exatamente o comportamento desejável no contexto hospitalar. Contra o baseline aleatório (443,79 km), o AG reduz a distância em **~46%**.

6. Integração com LLM (Claude)

A integração usa o SDK oficial da Anthropic (modelo `claude-sonnet-4-6`, configurável). A chave (`ANTHROPIC_API_KEY`) é lida **exclusivamente** de variável de ambiente; em produção, vem do **Azure Key Vault** via *Managed Identity* — nunca fica em código ou em commits.

São três funções, cada uma com *prompt* de sistema próprio:

1. **Instruções por motorista** (`generate_driver_instructions`) — gera um roteiro em português, parada a parada, com alerta destacado para medicamentos críticos e lembrete de retorno ao depósito.
2. **Relatório de eficiência** (`generate_efficiency_report`) — compara a rota otimizada com o baseline aleatório e produz um parágrafo executivo para gestores não técnicos (distância, economia em km e %, nº de críticos).
3. **Perguntas e respostas** (`answer_question_about_routes`) — responde em linguagem natural a perguntas como "qual entrega é mais urgente?" ou "qual veículo está mais sobrecarregado?", injetando no contexto um resumo de carga, autonomia e nº de críticos por veículo.

Engenharia de prompt. Cada *prompt* de sistema fixa o papel (assistente de logística, analista, assistente de Q&A), o idioma (português do Brasil) e o tom (objetivo para motoristas; executivo para gestores). Os dados das rotas são injetados de forma estruturada no *prompt* do usuário, reduzindo alucinação e mantendo as respostas ancoradas na saída real do otimizador.

Avaliação de qualidade. A qualidade é avaliada por: (i) **precisão factual** — a resposta reflete os dados injetados (ordem, pesos, prioridade)? (ii) **adequação ao público** — linguagem operacional vs. executiva; (iii) **segurança** — não inventa endereços nem reordena entregas críticas. Como o enunciado não exige latência mínima, priorizamos qualidade textual sobre velocidade.

7. Arquitetura e infraestrutura

A aplicação é um app **Streamlit** containerizado, com a seguinte estrutura de código:

```
app/
├─ main.py           # Interface Streamlit
├─ geocoding.py      # Nominatim (OpenStreetMap), gratuito
├─ genetic/          # AG: algorithm, operators, fitness
├─ llm/claude_client # Integração Claude
├─ maps/route_map    # Mapa Folium
└─ models/delivery   # Entidades do domínio (VRP)
```

Nuvem (opcional, implementado para pontuação extra). Toda a infraestrutura está descrita como código (**IaC com Bicep**) e provisiona, no Azure: Container Apps (1–3 réplicas com *autoscaling*), Container Registry, Key Vault, *Managed Identity*, Log Analytics. Decisões de segurança: sem *admin*

user no registry, segredos só no Key Vault via RBAC, *purge protection* e *soft delete* ativados. Um *pipeline* de **CI/CD (GitHub Actions)** roda os testes e, se passarem, faz *build* e *deploy* automático.

Qualidade de engenharia. O projeto tem **35 testes automatizados** (pytest) cobrindo operadores, fitness, decodificação, modelos, geocodificação (mockada) e cliente Claude (mockado, sem custo de API). O ambiente é gerenciado por **Poetry**.

8. Considerações éticas

Embora o cenário seja sintético, um sistema real de roteamento na saúde manipula dados sensíveis e exige cuidado ético:

- **Privacidade de localização.** Endereços de entrega podem revelar onde pacientes moram e que tratamento recebem (um endereço marcado como "medicamento crítico" é um dado de saúde indireto). Em produção, esses dados exigem minimização, controle de acesso e conformidade com a **LGPD**.
 - **Segurança dos dados.** A arquitetura já adota *Managed Identity* e Key Vault para segredos; o mesmo rigor deve valer para os dados de entrega (criptografia em trânsito e em repouso, retenção mínima).
 - **Viés e equidade.** Uma função fitness que otimiza apenas distância pode, de forma não intencional, **desfavorecer regiões periféricas ou de difícil acesso**, que ficariam sistematicamente no fim das rotas. A penalidade de prioridade mitiga isso para casos críticos, mas uma auditoria de equidade geográfica é recomendável antes de uso real.
 - **Autonomia profissional.** As instruções geradas pelo LLM são um **apoio à decisão**, não uma ordem. A equipe de logística e os profissionais de saúde mantêm a palavra final; o sistema não deve automatizar decisões clínicas.
-

9. Limitações e trabalhos futuros

Limitação	Impacto	Evolução possível
Distância em linha reta (Haversine)	Ignora vias, trânsito, sentido	Integrar OSRM / Google Directions
Decodificação <i>greedy</i>	Divisão entre veículos não é ótima	<i>Split</i> ótimo por programação dinâmica
Sem janelas de tempo	Não modela horário de entrega	Adicionar <i>time windows</i> ao fitness
Geocodificação a ~1 req/s (Nominatim)	Lenta com muitas entregas	Cache local / provedor pago
Estado em memória (Streamlit)	Recarregar perde os dados	Persistência (PostgreSQL / Supabase)

10. Conclusão

O MedRoutes cumpre integralmente os requisitos do Projeto 2: um **Algoritmo Genético próprio** com seleção, crossover, mutação e elitismo; **três restrições** (prioridade, capacidade, autonomia); **visualização em mapa**; **integração com LLM** para instruções, relatórios e Q&A; e uma **avaliação experimental com rigor estatístico** (10 sementes, média $\pm$ desvio-padrão) comparando quatro configurações e dois baselines.

O principal resultado técnico é que a configuração de **população média com mutação alta (Config C)** oferece o melhor equilíbrio entre qualidade (fitness 330,5), estabilidade ($\pm$ 10,8) e custo (0,39 s). O principal resultado conceitual é a constatação — só visível graças à análise multiobjetivo e ao baseline do vizinho-mais-próximo — de que **minimizar distância pura não é o objetivo certo** num contexto hospitalar: o AG troca alguns quilômetros por uma priorização clínica adequada, e essa é precisamente a decisão que o problema pede.

Referências

- Goldberg, D. E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- Holland, J. H. *Adaptation in Natural and Artificial Systems*. MIT Press, 1992.
- Toth, P.; Vigo, D. *Vehicle Routing: Problems, Methods, and Applications*. SIAM, 2014.

- Anthropic. *Claude API Documentation*. <https://docs.anthropic.com>
- OpenStreetMap Foundation. *Nominatim Usage Policy*.
<https://operations.osmfoundation.org/policies/nominatim/>